

ExCP: Extreme LLM Checkpoint Compression via Weight-Momentum Joint Shrinking

Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, Yunhe Wang

Huawei Noah's Ark Lab, University of Science and Technology of China

Background: Foundation models are transforming the world

- LLMs power search, assistants, coding, science—shaping products and research.
- Scale drives capability: billions to trillions of parameters improve accuracy and reasoning.
- Training stability needs frequent checkpointing for reliability and iteration.

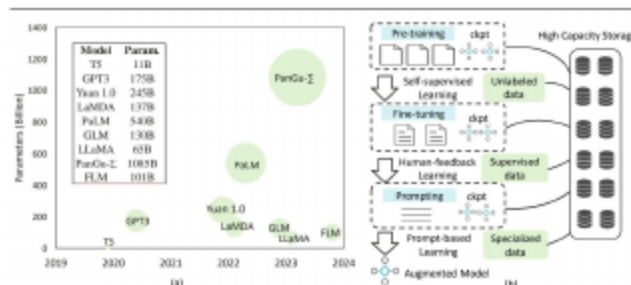


Figure: Parameter scales of recent LLMs and a high-level training workflow. Multiple checkpoints per stage drive high-capacity storage needs.

The Problem: Checkpoint storage is a scaling bottleneck

- Each checkpoint stores weights plus optimizer states (Adam moments dominate).
- Hundreds of checkpoints → multi-TB storage and high I/O overhead.
- Costs and bandwidth limit training cadence, safety, and experimentation.

Why existing approaches fall short

- Lossless codecs: minimal gains on floating-point tensors.
- Model pruning/quantization: target inference, not training states; ignore optimizer.
- Naïve lossy compression breaks resume fidelity and convergence guarantees.

Core idea and contributions (overview)

- ExCP: compress checkpoints via residual storage, joint shrinking with momentum, and non-uniform quantization.
- Exploit temporal sparsity and optimizer statistics to prune safely.
- Near-lossless resume and continued training with 25–70× storage reduction.

Method overview: ExCP compression pipeline

- Residuals: compute $\Delta W^t = W^t - \hat{W}^{t-1}$ against last reconstructed weights to exploit temporal sparsity and isolate essential updates.
- Joint masks: couple weight and momentum pruning so orphan moments are avoided and key optimization signals are preserved.
- Non-uniform quantization: K-means codebooks for nonzeros; explicit zero; store centers and low-bit indices.
- Entropy coding and reconstruction: compress indices/centers; resume by decoding ΔW^t and adding to $\hat{W}^{t-1}$.

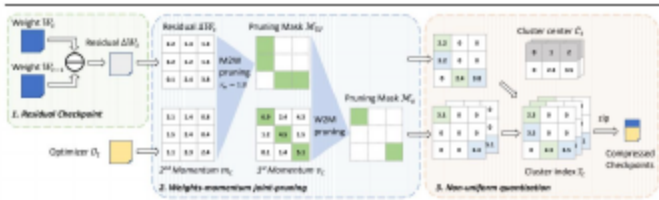


Figure: Framework of the ExCP compression process: residuals, joint pruning, non-uniform quantization, and entropy coding.

ExCP Checkpoint Compression

Component 1: Residual checkpoints enable safe sparsification

- Adjacent checkpoints differ slightly—temporal sparsity.
- Store $\Delta W^t = W^t - \hat{W}^{t-1}$ to isolate essential updates.
- Residuals preserve distributional shape, enabling higher pruning ratios.

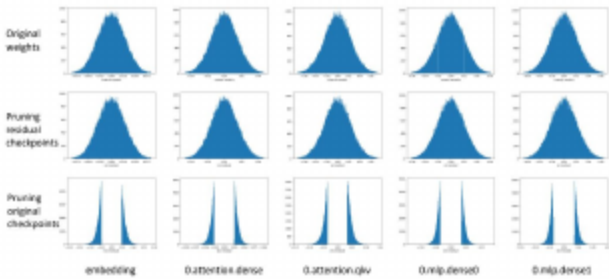


Figure: Weight distributions under three settings: original weights, pruning on residual checkpoints, and pruning on original weights. Histograms use 256 bins within mean $\pm 3 \times \text{std}$.

ExCP Checkpoint Compression

Component 2: Weight–momentum joint shrinking

- Weight pruning threshold per layer uses second-order stats (adaptive per-layer).
- Momentum pruning is conditioned on weight masks to avoid orphan moments.
- Coupling discards redundant states while preserving critical optimization signals.

ExCP Checkpoint Compression

Component 3: Non-uniform quantization and archival

- Cluster nonzeros with K-means; keep zeros explicit.
- Separate codebooks per tensor type (e.g., weights, moments); 4-bit indices commonly used for efficiency.
- Pack int4 efficiently; store cluster centers plus index maps.
- 4-bit typically provides the best size–accuracy trade-off vs. 2-bit and 8-bit.
- Final archival with 7zip/LZMA2 is effective.
- Separate codebooks per tensor type improve efficiency.

Compression–reconstruction protocol

- During training: reconstruct $\hat{W}^{t-1}$, compute ΔW^t , prune+quantize, archive.
- To resume: decode centers/indices $\rightarrow$ quantized $\Delta W^t \rightarrow$ add to $\hat{W}^{t-1}$.
- Arbitrary checkpoint recovery: apply residuals sequentially from seed.

Convergence and training guarantees

- Extends Adam analysis with masks on moments at pruning steps.
- Regret bound adds small term tied to pruned second-moment magnitudes.
- Average regret remains $O(1/\sqrt{T})$; pruning targets small entries to limit impact.

Implementation details

- Defaults: $\alpha = 5e-5$, $\beta = 2.0$; 4-bit quantization with int4 packing (two int4 per int8); 7zip/LZMA2 archival.
- LLMs save every 1000 iters; optimizer states handled per component.

- Models: ViT-L/32; Pythia-410M; PanGu- π 1B and 7B.
- LLM evals: HellaSwag, ARC-e, PIQA, C3, CSL, LAMBADA; PanGu per prior protocols.
- Goal: verify storage reduction, resume fidelity, and downstream quality.

- Frontier LLMs require TB-scale checkpoints per save.
- Adam moments roughly double weight storage needs.
- Frequent saves across runs translate to multi-TB (or more) footprints.
- Examples: $\sim 175\text{B}$ parameters $\approx 2.3\text{ TB}$ per checkpoint; $\sim 540\text{B} \approx 7\text{ TB}$.

Vision baseline: ViT-L/32 shows momentum compression matters

- ExCP attains $35.21\times$ (weights+momentum) with slight accuracy gains.
- Prior methods focus on weights; $< 1.5\times$ when momentum is included.
- Joint pruning is a key differentiator for training-state compression.

Asterisks indicate effective CR when accounting for optimizer moments.

Results on ViT-L/32 (ImageNet-1K). CR denotes compression ratio.

Method	M2W pruning	W2M pruning	Top-1 Accuracy(%)	CR(Weights)	CR(Weights & Momentum)
baseline			71.36	1	1
CNR+			71.57	7.82	1.41*
QD+			71.24	16.31	1.45*
DynaQuant			71.82	26.19	1.47*
ExCP (Ours)			71.51	-	19.88
ExCP (Ours)	X		71.53	-	25.54
ExCP (Ours)		X	71.80	-	22.76
ExCP (Ours)	X	X	71.69	-	35.21

LLM case study: Pythia-410M achieves $\sim 70\times$ compression

- Storage shrinks from 4.53 GB to 0.06 GB per checkpoint.
- Downstream average remains virtually unchanged.
- They observe slight improvements on some tasks, suggesting a regularization effect.

Pythia-410M: downstream accuracy and checkpoint size.

Model	Method	Size	Tasks						Avg
			hellaswag	arc-e	piqa	C3	cs1	lambada	
Pythia-410M	Original model	4.53G	32.52	35.80	62.13	37.21	53.75	37.22	43.11
	Residual+7Zip	3.40G	32.52	35.80	62.13	37.21	53.75	37.22	43.11
	ExCP (Ours)	0.06G	31.95	37.04	62.62	36.22	52.50	37.24	42.93

- Loss curves nearly overlap with original training.
- Per-checkpoint size correlates with learning rate via residual magnitude.
- This supports the residual rationale and indicates low training perturbation.

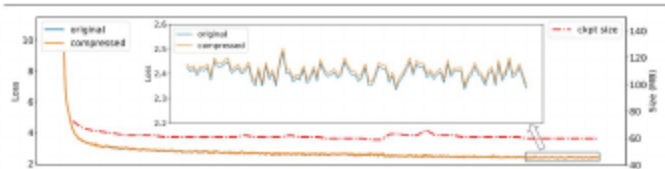


Figure: Comparison of training loss and checkpoint size between the original run and ExCP.

- Achieves $\sim 25\times$ compression with similar downstream averages.
- Qualitative behavior aligns with baselines at comparable training stages.
- Confirms scalability from 410M to multi-billion parameters.

Ablation: Residuals, joint pruning, and quantization work together

- Residualization alone reduces size modestly while maintaining accuracy.
- Pruning without residuals harms accuracy; residuals enable safe sparsity.
- The best trade-off emerges from combining residuals, joint pruning, and quantization.

Ablation: effect of residuals, joint pruning, and quantization.

residual	prune	quant	Size	Avg Acc		
X	X	X	4070M	43.11		
			3484M	43.11		
			324M	29.95		
X	X	X	492M	40.17		
			276M	42.92		

Practical benefits: Cost, cadence, and safety

- 25–70 $\times$ fewer bytes $\rightarrow$ lower cloud bills and emissions.
- Faster I/O enables more frequent and safer checkpointing.
- Decoupled design integrates without training code changes.

Limitations and cautions • Sequential reconstruction for arbitrary checkpoints adds latency post-hoc. • Hyperparameters (α, β, save frequency) may need tuning per regime. • Assumes Adam-like optimizers; other optimizers may require adaptation. ExCP Checkpoint Compression	Future work and broader impact • Future directions include extensions to multimodal and visual LLMs and other architectures. • Adaptive schedules for pruning/quantization over training are promising. • Potential to lower barriers to experimentation and enable greener large-scale training. ExCP Checkpoint Compression
Open Questions • How robust is joint pruning under non-Adam optimizers and different momentum schedules? • Can residual sparsity be adaptively scheduled with LR warmup/decay phases? • What are the limits of long reconstruction chains on failure recovery time? • How to combine ExCP with distributed checkpointing/fault tolerance? • Can content-adaptive codebooks be shared across layers to reduce metadata? • What monitoring signals best flag unsafe sparsification during training? ExCP Checkpoint Compression	Thank you • Thank you for your attention! ExCP Checkpoint Compression